

Contents

1. Reverse a String using Stack	2
Algorithm	2
2. Balanced Parentheses.....	2
Algorithm	3
3. Infix to Postfix Conversion.....	3
Algorithm	3
4. Infix to Prefix.....	4
Algorithm	4
5. Postfix Evaluation.....	4
Algorithm	5
6. Prefix Evaluation	5
7. Next Greater Element	5
Algorithm	6
8. Next Smaller Element.....	6
9. Previous Greater Element	6
10. Previous Smaller Element.....	7
11. Largest Rectangle in Histogram	7
12. Stock Span Problem.....	7
13. Celebrity Problem.....	8
14. Tower of Hanoi.....	8
15. Reverse Stack	8
16. Sort Stack using Stack	9
17. Remove Middle Element of Stack	9
18. Reverse First K Elements using Stack	9
19. Implement Stack using Queue	9
20. Implement Queue using Stack	10
21. Minimum Stack (Min Stack)	10
22. Maximum Stack.....	10
23. Undo/Redo Operation.....	10
24. Browser History.....	11
25. Function Call Stack.....	11
26. Depth First Search (DFS)	11
27. Expression Tree Construction.....	11
28. Decode String	12
29. Remove Adjacent Duplicates.....	12
30. Asteroid Collision.....	12

31. Daily Temperatures	12
32. Valid Parenthesis String	13
33. Simplify Unix Path	13
34. Decode Infix Expression.....	13
35. Check Palindrome using Stack	13
36. Remove Consecutive Characters	14
37. Reverse Words in Sentence	14
38. Check Redundant Brackets	14
39. Next Greater Frequency Element	14
40. Trapping Rain Water (Stack Approach)	14
Most Frequently Asked Stack Problems in Interviews	14

Stack-based algorithms asked in DSA interviews

(TCS, Infosys, Wipro, Accenture, Amazon, Microsoft, Google, Oracle, etc.)

1. Reverse a String using Stack

Algorithm

Step 1: Read the string
Step 2: Push every character into stack
Step 3: Pop characters one by one
Step 4: Append popped character into new string
Step 5: Display reversed string

Time Complexity: $O(n)$

2. Balanced Parentheses

Example

Input:
{ [(())] }

Output:
Balanced

Algorithm

Step 1: Create empty stack

Step 2:
Traverse every character

Step 3:
If opening bracket
Push it

Step 4:
If closing bracket

If stack empty
Not Balanced

Else

Pop top

Check matching bracket

If mismatch
Not Balanced

Step 5:
If stack empty
Balanced

Else
Not Balanced

Complexity

Time : $O(n)$
Space : $O(n)$

3. Infix to Postfix Conversion

Example

A+B*C

Output

ABC*+

Algorithm

Step 1:
Create empty stack

Step 2:
Scan expression left to right

Operand
Print

Opening bracket
Push

Closing bracket
Pop until opening bracket

Operator

While stack precedence $\geq$ current

Pop

Push current operator

Step 3

Pop remaining operators

4. Infix to Prefix

$(A+B) * C$

Output

*+ABC

Algorithm

Reverse expression

Exchange brackets

Convert into postfix

Reverse answer

5. Postfix Evaluation

Example

23*54*+

Output

26

Algorithm

Scan expression

Operand

Push

Operator

Pop operand2

Pop operand1

Calculate

Push result

Display top element

6. Prefix Evaluation

+9*26

Output

21

Algorithm

Scan from Right

Operand

Push

Operator

Pop two operands

Calculate

Push result

Print final answer

7. Next Greater Element

Example

4 5 2 25

Output

5 25 25 -1

Algorithm

Push first element

Traverse array

While current > stack top

Print next greater

Pop

Push current

Remaining

Print -1

Complexity

$O(n)$

8. Next Smaller Element

Example

4 8 5 2

Output

2 5 2 -1

Uses **Monotonic Stack**

9. Previous Greater Element

Example

4 2 7 3

Output

-1 4 -1 7

10. Previous Smaller Element

4 5 2 8

Output

-1 4 -1 2

11. Largest Rectangle in Histogram

Example

2 1 5 6 2 3

Output

10

Uses stack for left and right boundaries.

Complexity

$O(n)$

12. Stock Span Problem

Example

100 80 60 70 60 75 85

Output

1 1 1 2 1 4 6

Algorithm

Push index

Traverse

While current price $\geq$ stack top

Pop

Calculate span

Push index

Complexity

$O(n)$

13. Celebrity Problem

Find celebrity in party

Algorithm

Push everyone

Pop two

Remove one

Repeat

Verify remaining person

Complexity

$O(n)$

14. Tower of Hanoi

Uses recursion (implicit stack).

Algorithm

Move $n-1$

Move largest disk

Move $n-1$

Complexity

$O(2^n)$

15. Reverse Stack

Algorithm

Pop

Reverse remaining

Insert at bottom

16. Sort Stack using Stack

Algorithm

Create auxiliary stack

Pop element

Insert in sorted order

Move back

17. Remove Middle Element of Stack

Algorithm

Pop recursively

Skip middle

Push back

18. Reverse First K Elements using Stack

Queue + Stack

Queue

↓

Stack

↓

Queue

19. Implement Stack using Queue

Operations

Push

Pop

Peek

isEmpty

20. Implement Queue using Stack

Operations

Enqueue

Dequeue

Peek

21. Minimum Stack (Min Stack)

Supports

Push

Pop

Top

Get Minimum

All operations in **O(1)**.

22. Maximum Stack

Supports

Push

Pop

Get Maximum

23. Undo/Redo Operation

Uses **Two Stacks**

Undo Stack

Redo Stack

Applications

- Text Editors
- IDEs

- Paint Software

24. Browser History

Uses two stacks

Back

Forward

25. Function Call Stack

Every recursive function call is stored in the stack.

Example

Factorial

fib()

Tower of Hanoi

DFS

26. Depth First Search (DFS)

Uses stack.

Algorithm

Push source

While stack not empty

Pop

Visit

Push neighbours

27. Expression Tree Construction

Postfix

↓

Expression Tree

Uses stack of nodes.

28. Decode String

Example

`3[a2[c]]`

Output

`accaccacc`

Uses two stacks.

29. Remove Adjacent Duplicates

Example

`abbaca`

Output

`ca`

Uses stack.

30. Asteroid Collision

Example

`5 10 -5`

Output

`5 10`

Uses stack.

31. Daily Temperatures

Example

73 74 75 71 69 72 76 73

Output

1 1 4 2 1 1 0 0

Uses monotonic stack.

32. Valid Parenthesis String

(*))

Output

True

Uses stacks for (and *.

33. Simplify Unix Path

Example

/a/./b/../../../../c/

Output

/c

Uses stack.

34. Decode Infix Expression

(A+B) * (C-D)

Uses operator and operand stacks.

35. Check Palindrome using Stack

Algorithm

Push first half

Compare with second half

36. Remove Consecutive Characters

Example

aaabbccdde

↓

abcde

37. Reverse Words in Sentence

I Love Java

↓

Java Love I

38. Check Redundant Brackets

Example

((a+b))

↓

Redundant

39. Next Greater Frequency Element

Uses stack + frequency map.

40. Trapping Rain Water (Stack Approach)

Height array

↓

Water trapped

Complexity

$O(n)$

Most Frequently Asked Stack Problems in Interviews

Problem

Difficulty Company Frequency

Problem	Difficulty	Company Frequency
Balanced Parentheses	Easy	★★★★★
Infix to Postfix	Easy	★★★★★
Postfix Evaluation	Easy	★★★★★
Stock Span	Medium	★★★★★
Next Greater Element	Medium	★★★★★
Largest Rectangle in Histogram	Hard	★★★★★
Min Stack	Medium	★★★★★
Celebrity Problem	Medium	★★★★☆
Undo/Redo	Medium	★★★★☆
Browser History	Medium	★★★★☆
Queue using Stack	Easy	★★★★★
Stack using Queue	Easy	★★★★★
DFS using Stack	Medium	★★★★★
Simplify Unix Path	Medium	★★★★☆
Daily Temperatures	Medium	★★★★★

These algorithms collectively cover the vast majority of stack-related concepts encountered in technical interviews and competitive programming. Mastering them provides a strong foundation for solving more advanced stack-based problems.